Comparison Analysis of CPU Scheduling Algorithms: Round Robin, FCFS, SJF, Priority Based

Mayank Shekhar Pandey
School of Computer Science and Engineering
Lovely Professional University
Phagwara,India
mayankpandey3009@gmail.com

***Abstract*— This paper discusses various aspects of CPU scheduling. As it is a fundamental function of an operating system (OS) that directly impacts system efficiency and multitasking performance. A robust scheduling algorithm can ensure optimal CPU utilization while minimizing process wait times, turnaround times, and context switches. Scheduling criteria, the optimization of which is the ultimate goal of a CPU scheduling algorithm. This paper presents a comparative study on widely used CPU scheduling algorithms: First-Come-First-Serve (FCFS), Shortest Job First (SJF), Round Robin (RR), and Priority Scheduling. Each algorithm offers distinct advantages and is suited to different workload scenarios. Round Robin algorithm, known for its fairness in time-sharing systems, assigns equal time quanta to processes, making it highly suitable for time-sensitive, multitasking environments.**

FCFS is straightforward, assigning CPU time to processes in arrival order. However, its simplicity can lead to high turnaround times in mixed workloads. SJF, which prioritizes shorter jobs, can achieve lower turnaround times but risks process starvation for longer tasks. Priority Scheduling addresses this by assigning priority levels to processes. Advantages and disadvantages of each of these algorithms are also explored. Simulations highlight the performance of these algorithms under various workloads, providing insights into each algorithm's ideal usage scenarios. The study concludes that while no single scheduling approach is universally optimal, adaptive and hybrid algorithms can significantly improve OS performance.

Keywords—CPU scheduling, Round Robin CPU scheduling algorithm, Operating System, FCFS, SJF, SRTF, Priority based, Multilevel queuing, Turnaround time, Waiting time, Response time

I. Introduction

CPU scheduling is a core function in operating systems, facilitating efficient management of processes by allocating CPU time, the most vital system resource, among various active tasks. When a user working at a computer, requests a program, it first checks whether it can be found in the random access memory (RAM). If it is not found in the RAM, it has to be fetched from permanent storage such as the hard disk, where a copy of the requested program will be created and brought to the RAM. By determining the sequence and duration for which each process accesses the CPU, scheduling optimizes system performance, ensuring processes are executed in an orderly and efficient manner. This scheduling becomes especially critical in multitasking environments, where multiple processes are competing for CPU time. Each scheduling method aims to improve system metrics like turnaround time, waiting time, response time, and overall throughput.

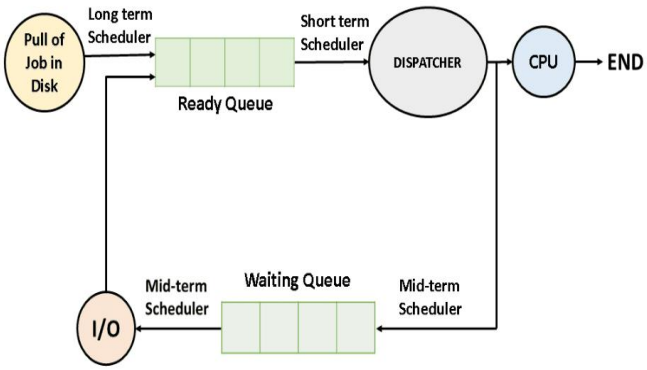

Fig.1. CPU scheduling functionality

Several CPU scheduling algorithms have been developed to optimize these performance metrics based on specific workload characteristics. The First-Come-First-Serve (FCFS) algorithm operates by serving processes in the order they arrive, offering simplicity but potentially leading to high waiting times for longer tasks. Shortest Job First (SJF) prioritizes tasks with the smallest CPU burst times, minimizing turnaround time but risking process starvation when longer tasks must wait.

The Round Robin (RR) algorithm, widely used in time-sharing systems, allocates a fixed time quantum to each process, rotating through the queue to ensure that no process is left waiting indefinitely. Though it improves responsiveness, the effectiveness of RR depends on the selection of an appropriate time quantum; a too-short quantum increases context switches, while a too-long one reintroduces high waiting times. Some enhanced versions of RR, such as Smart Round Robin (SRR), incorporate dynamic time quantum adjustments, addressing limitations by adapting to process needs.

Priority Scheduling assigns CPU access based on the priority level of each process, balancing performance and fairness according to task importance. However, this can lead to priority inversion, where lower-priority processes can significantly delay higher-priority ones unless mitigated through adjustments or a multilevel scheduling approach.

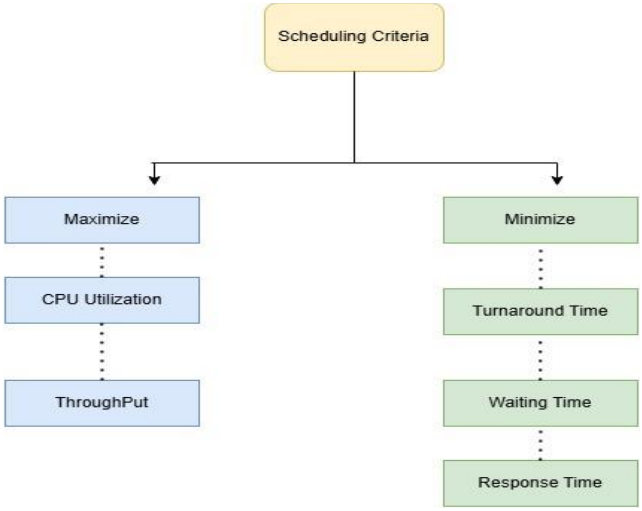

Fig.2. Scheduling Criteria

ideal application scenarios for these algorithms. By understanding the unique characteristics of each, system designers and operators can make informed choices, tailoring CPU scheduling strategies to meet specific system requirements and workload demands effectively.

II. CPU SCHEDULING ALOGITHMS

A. Round Robin Scheduling

In recent times different approaches have been used to increase the performance of CPU scheduling algorithms. In the given algorithm, the processes are arranged in increasing order of their burst times. The modified time quantum is calculated based on the number of processes. The time quantum is applied to all processes. Figure 1. represents the Modified Round Robin algorithm steps. The advantage of this method is that the time quantum more efficiently rolls around the processes reducing its waiting time. The drawback is that for every cycle the time quantum is static so it does not provide dynamicity for each cycle.

The RR algorithm treats the order of processes in the ready queue in that new processes get added to the end of the ready queue. The main distinction is the existence of a time quantum (TQ) sometimes referred to as a time slice, after which time if the BT of current process is larger than TQ, the current process is pre-empted and the next process is given the CPU access. If the current process has a less BT than TQ, at CT of the current process, the next process in queue will be given access. If the TQ is too large relative to the BTs of the processes. If the TQ

is too small when compared to BTs, there will be many context switches which adversely affects the performance of the process. The value of TQ can be either static or dynamic. Although having a static TQ is good for equal time sharing, some processes which are highly interactive when compared to others, need a relatively larger TQ and therefore having numerous context switches can become cumbersome. This can be overcome by having a dynamic TQ which gets updated according to the BTs of the previous processes.

Advantages- It gives the best performance in terms of average response time. It is best suited for time sharing system, client server architecture and interactive system.

Disadvantages- It leads to starvation for processes with larger burst time as they have to repeat the cycle many times. Its performance heavily depends on time quantum. Priorities cannot be set for the processes.

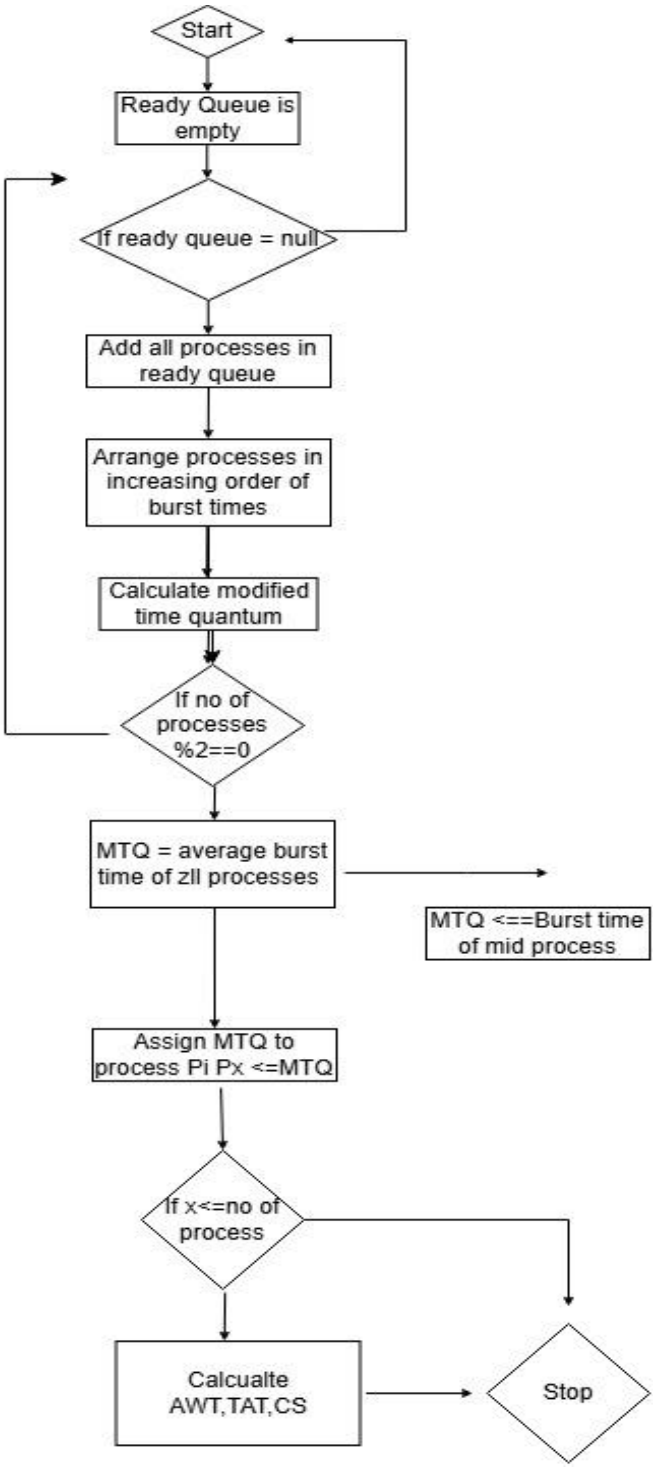

Fig.3. Flow Chart smart Round Robin

Table.1. Turnaround time and waiting time calculation for RR

P	AT	BT	CT	TAT	WT
A	0	5	20	20	15
B	1	3	13	12	9
C	2	4	19	17	13
D	3	6	28	25	19
E	4	2	12	8	6
F	5	5	29	24	19
G	6	7	32	26	19

Average TAT = (20+12+17+25+8+24+26)/7 = 18.8571

Average WT = (15+9+13+19+6+19+19)/7 = 14.2857

B. Priority Based

In PB scheduling, there is a numerical value assigned to each process which denotes the priority ranking of that process. Depending on the context, a small numerical value might mean a higher priority or a lower priority. Higher priority processes will be given CPU access earlier when compared to lower priority processes. SJF and SRTF can be considered as versions of PB algorithm where the numerical figure that governs priority are either CPU burst time or the remaining CPU burst time. Just like with SJF and SRTF, there is a risk of low priority processes getting starved (i.e. being held up in the ready queue without CPU access for longer times) if there is a long string of high priority processes getting added to the ready queue. One method to overcome starvation is 'aging' where the priority of a process is gradually increased with its waiting time in the queue. PB scheduling can be either pre-emptive or non-pre-emptive. Application of non-pre-emptive PB on processes in TABLE results in TATs and WTs given by TABLE . Fig. shows the corresponding Gantt chart.

Average T AT = (5 + 7 + 10 + 24 + 10 + 27 + 15)/7 = 14

Average W T = (0 + 4 + 6 + 18 + 8 + 22 + 8)/7 = 9.4286

Table.2. Turnaround time and waiting time calculation for PB

P	AT	BT	CT	TAT	WT
A	0	5	5	5	0
B	1	3	8	7	4
C	2	4	12	10	6
D	3	6	27	24	18
E	4	2	14	10	8
F	5	5	32	27	22
G	6	7	21	15	8

Advantages- It considers the priority of the processes and allows the important processes to run first. Priority scheduling in pre-emptive mode is best suited for real time operating system.

Disadvantages- Processes with lesser priority may starve for CPU. There is no idea of response time and waiting time.

C. First Come First Served

This is largely considered as the simplest CPU scheduling algorithm. The processes in the ready queue are given CPU access based on their arrival time at the ready queue. Processes that arrive earlier will get CPU access earlier than the processes that arrive later at the ready queue. This is a non-pre-emptive algorithm, meaning that once the CPU starts executing a process, until that process finishes, CPU access will not be given to another process. If a particular process has a large CPU burst time, all other processes will have to wait for longer times in the ready queue, which will lead to poor CPU utilization, poor throughput, large average turnaround times, and large average waiting times. This drawback of FCFS where processes with shorter CPU burst times are being held up due to CPU being occupied with processes with large CPU burst times is known as 'convoy effect'. The TABLE below shows arrival times (AT), burst times (BT), and priorities for seven processes in the ready queue.

Table.3. Information About Processes

P	AT	BT	Priority
A	0	5	3
B	1	3	5
C	2	4	7
D	3	6	11
E	4	2	9
F	5	5	13
G	6	7	10

The corresponding completion times (CT), turnaround times (TAT), and waiting times (WT) are shown in TABLE . We have assumed that the time taken for context switching is zero. We have used the relationships T AT = CT − AT and W T = T AT −BT. Fig. shows the corresponding Gantt chart.

Table.4. Turnaround time and waiting time calculation for FCFS

P	AT	BT	CT	TAT	WT
A	0	5	5	5	0
B	1	3	8	7	4
C	2	4	12	10	6
D	3	6	18	15	9
E	4	2	20	16	14
F	5	5	25	20	15
G	6	7	32	26	19

Average T AT = (5 + 7 + 10 + 15 + 16 + 20 + 26)/7 = 14.1429

Average W T = (0+ 4+ 6+ 9+ 14+ 15+ 19)/7 = 9.57143

Advantages- It is simple and easy to understand. It can be easily implemented using queue data structure. It does not lead to starvation.

Disadvantages- It does not consider the priority or burst time of the processes. It suffers from convoy effect i.e. processes with higher burst time arrived before the processes with smaller burst time.

D. First Come First Served

This algorithm overcomes the convoy effect of FCFS by giving earlier CPU access to processes with shorter BT. This can be either pre-emptive or non-pre-emptive. The pre-emptive version is also known as the Shortest Remaining Time First (SRTF) algorithm. In SRTF, if a new process arrives while the current process is being executed by the CPU, and if the new process has a shorter BT than the remaining part of the current process, the current process will be pre-empted and the new process will be given CPU access. In the non-pre-emptive SJF algorithm, the process which arrived first will be given CPU access first and it will run till its completion. Out of the processes that have arrived by the CT of the first process, the job with the shortest BT will be given access to CPU and this pattern will continue. The application of non-pre-emptive SJF for the processes in TABLE , result in TAT values and WT values given in TABLE . Fig. shows the corresponding Gantt chart.

Table.5. Turnaround time and waiting time calculation for SJF

P	AT	BT	CT	TAT	WT
A	0	5	5	5	0
B	1	3	10	9	6
C	2	4	14	12	8
D	3	6	25	22	16
E	4	2	7	3	1
F	5	5	19	14	9
G	6	7	32	26	19

Average T AT = (5 + 9 + 12 + 22 + 3 + 14 + 26)/7 = 13

Average W T = (0 + 6 + 8 + 16 + 1 + 9 + 19)/7 = 8.42857

Advantages- It is optimal and guarantees the minimum average waiting time. It provides a standard for other algorithms since no other algorithm performs better than it.

Disadvantages- It cannot be implemented practically since burst time of the processes cannot be known in advance. It leads to starvation for processes with larger burst time. Priorities cannot be set for the processes. Processes with larger burst time have poor response time.

III. FUTURE SCOPE

Future work can be based on modifying and implementing the algorithm for multiprocessor systems and real time systems where processes might have deadlines and must be completed within that time constraint. In future, priority of each task can be considered as an additional information along with process ID, burst time and arrival time. Based on the priority, lower priority processes will be pre-empted if a high priority task requires CPU time.

This research has been defined as a preliminary step for scientists, because there are still some problems which can be cleared up and enhance in future works, including such (1) enhancing the RR algorithm by trying to find a novel paradigm for time quantum calculation that manages to combine dynamic and fixed quantum values to improve RR algorithm efficiency, and (2) enhancing others algorithm First-Come-First-Serve (FCFS), Shortest Job First (SJF) and Priority Scheduling.

IV. CONCLUSION

This article provides a gentle walkthrough regarding various characteristics of some of the most popular and widely used CPU scheduling algorithms with guided examples to compare the efficiency of the algorithms, some of their advantages, disadvantages and remedies. The field of CPU scheduling is rapidly evolving and sophisticated algorithms are being created by researchers at a robust pace. In this regard this article has provided key information to provide a good foundational knowledge about CPU scheduling to inquisitive minds. The main objective of CPU Scheduling is to improve the system efficiency.

This study proposes an improved version of the round robin scheduling technique in which it provides dynamic time quantum for each cycle and specifically helps tasks which have short remaining burst times. It focuses on computing the mean for all jobs/tasks in the ready queue, which is sorted using the SJF approach, in order to discover a solution to the time quantum problem. For each job/task and each round, the operation of dynamically adjusting the time quantum is repeated. Other than it, First-Come-First-Serve (FCFS) and Priority Scheduling technique algorithms improve utilization CPU reduce idle time resource.

REFERENCES

[1] A. Silberschatz, G. Gagne, and P. Galvin, Operating System

[2] A. A. AL-Bakhrani, A. A. Hagar, A. A. Hamoud, and S. Kawathekar,

[3] T. Aladwani, Types of Task Scheduling Algorithms in Cloud Computing

[4] W. Zhao and J. Stankovic, "Performance analysis of fcfs and improved fcfs scheduling algorithms for dynamic real-time computer systems," in [1989] Proceedings. Real-Time Systems Symposium, 1989, pp. 156–165

[5] H. Parekh and S. Chaudhari, "Improved round robin cpu scheduling algorithm: Round robin, shortest job first and priority algorithm coupled to increase throughput and decrease waiting time and turnaround time, 12 2016, pp. 184–187.

[6] Silberschatz, A., Peterson, J. L., and Galvin, B.,Operating System Concepts, Addison Wesley, 7th Edition, 2006.

[7] E.O. Oyetunji, A. E. Oluleye," Performance Assessment of Some CPU Scheduling Algorithms", Research Journal of Information Technology, 1(1): pp 22-26, 2009.

[8] N. Mittal, K. Garg, A. Ameria , (2015) "A Paper on Modified Round Robin Algorithm", IJLTEMAS, Volume 4, Issue 11, ISSN 2278 – 2540.

[9] A. Singh, P. Goyal, S. Batra, (2010) " An Optimized Round Robin Scheduling Algorithm for CPU Scheduling", (IJCSE) International Journal on Computer Science and Engineering, Vol. 02, Issue 7, 2383-2385..

[10] M. Ramakrishna , G. Pattabhi Rama Rao, (2013) "Efficient Round Robin CPU Scheduling Algorithm For Operating System " IJITR, Vol. 1 , Issue 1, 103-109

[11] H. Heidari dan A. Chalechale, “Scheduling in Multiprocessor System Using Genetic Algorithm,” International Journal of Advanced Science and Technology, vol. 43, pp. 81-94, 2012.

[12] C. Bajaj, A. Dogra dan G. Singh, “Review And Analysis Of Task Scheduling Algorithms,” International Research Journal of Engineering and Technology, vol. 2, no. 3, pp. 1449-1452, 2015

[13] N. Hamid, “Scheduling Policies,” [Online]. Available: http://homepages.uel.ac.uk/u8902383/scheduling_policies.htm.© 2016 IJEDR | Volume 4, Issue 3 | ISSN: 2321-9939 IJEDR1603022 International Journal of Engineering Development and Research

[14] Arpaci-Dusseau, “Shortest Job Next,” 12 April 2016. [Online]. Available: https://en.wikipedia.org/wiki/Shortest_job_next.[Diakses 1 May 2016]

[15] A. Dusseau, R. H. dan A. C., Operating Systems: Three Easy Pieces, Arpaci-Dusseau Books, 2014

[16] K. Dev and S. Reda, ”Scheduling challenges and opportunities in integrated CPU+GPU processors,” 2016 14th ACM/IEEE Symposium on Embedded Systems for Real-time Multimedia (ESTIMedia), Pittsburgh, PA, USA, 2016, pp. 1-6.

[17] Md. Mamunur Rashid and Md. Nasim Adhtar; “A New Multilevel CPU Scheduling Algorithm”, Journals of Applied Sciences 6 (9): 2036-2039,2009.

[18] S. Huajin’, G. Deyuan, Z. Shengbing, W. Danghui;”Design Fast Round Robin Scheduler in FPGA”, 0-7803-7547-5/021/17.00 @ 2002 IEEE.

[19] S.Panda, S.Bhoi , “An Effective Round Robin Algorithm using MinMax Dispersion Measure,” International Journal on Computer Science and Engineering, vol. 4, no. 1, pp. 45-53, 2012.

[20] S. Hiranwal, Dr. K.C. Roy, “Adaptive Round Robin scheduling using shortest burst approach based on smart time Slice”, International Journal of Computer Science and Communication, Vol 2 (2), pp. 319-323, (JulyDecember) 2021